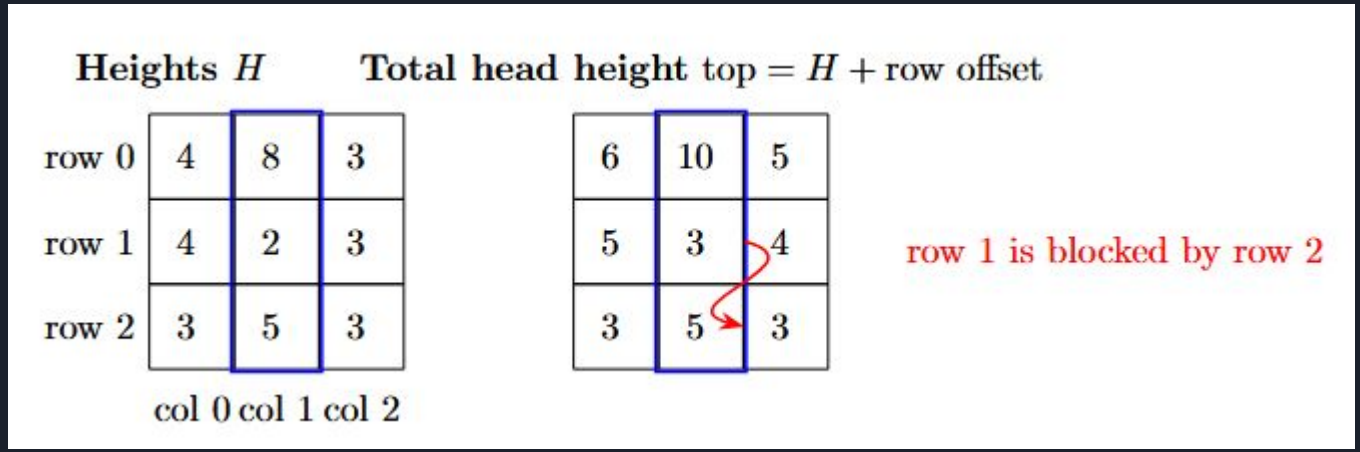
No Hats in the Cinema

Who's blocking my view?

Given a theatre grid of heights, for each seat find the closest person in front who blocks that seat.1

- **Grid H with R rows, C columns of integers**
- **H[i][j] = the height of the person sitting at row i column j**
- **Let Top(i, j) = H[i][j] + (i - 1).**
- **Person (i, j) blocks Person (k, j) if top(i, j) ≥ top(k, j)**
- **For each seat, output the index of the closest blocking person in front of it, or -1 if unblocked.**

Sample Input 1:



Naive Solution: Brute Force

- For each seat, check if that person can see the screen.

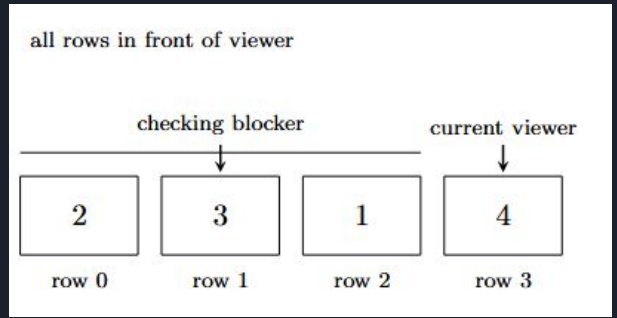
- Check visibility by looking at every person in front of them in the same column. If any person in front is tall enough, then they are blocked. Else unblocked.

```
blocker = array R x C filled with -1   // nearest blocker row for each seat

for c from 0 to C-1:
    for r from 0 to R-1:
        if H[r][c] == -1:
            continue  // empty seat

        // scan rows in front of r, nearest first
        for k from r-1 down to 0:
            if H[k][c] == -1:
                continue  // empty seat

            // if row k blocks row r (using problem's condition)
            if blocks(H[k][c], k, H[r][c], r):
                blocker[r][c] = k     // closest blocking row
                break                 // stop at first (nearest) blocker
```



Naive Solution: Brute Force

Time Complexity:

- For each column and for each row, scan all rows.
- In the best case, the scan is O(1), and in the worst case, the scan is linear.
- With **R** rows and **C** columns, that's **$O(R^2C)$** in the worst case and **O(RC)** in the best case.

TLEs on xyz/abc testcases:

- On large inputs when R, C ~= 10^4, $O(R^2C)$ is too slow and times out on the hidden testcases.

Space Complexity:

- Constant space besides the output grid, so O(1)

Optimal Solution: Nearest Blocker with Monotonic Stack

- Still handle columns independently
- Maintain a stack of indices of possible blockers.
- For each person/row, pop from the stack until the head is tall enough to block.
- Add the person to the stack

```
for c from 0 to C-1:
    stack = empty

    for r from 0 to R-1:
        if H[r][c] == -1:
            blocker[r][c] = -1
            continue

        eff_r = effectiveHeight(r, c)

        // remove candidates that cannot block r
        while stack not empty:
            k = stack.top()
            eff_k = effectiveHeight(k, c)
            if blocks(eff_k, eff_r):
                break
            stack.pop()

        if stack empty:
            blocker[r][c] = -1
        else:
            blocker[r][c] = stack.top()
        stack.push(r)
```

Optimal Solution: Nearest Blocker with Monotonic Stack

Time Complexity:

- For each column, each row gets pushed & popped at most once.
- In every case, processing a cell is amortized O(1)
- With **R** rows and **C** columns, **O(RC)**

Space Complexity:

- Maintain a stack of row indices.
- Best case is O(1) when increasing effective height, as stack always empty.
- Worst case is O(R) when decreasing effective height as stack contains [0,1...r-2]

Efficiency:

- O(RC) time is unavoidable. BF has constant space.
- R, C ~= $10^4 \Longrightarrow 10^8$, very tight fit for O(RC). Polynomial no. Log maybe.

Can be more efficiency?

- O(RC) time is unavoidable as we need to process every cell to construct our answer.
- Brute force is more space efficient since it's always constant space.

Test Suite & Cases

Test file contents:

N M

N lines with **M** space separated integers, either -1 or positive.

Edge Cases:

R = 1
C = 1
Everyone Blocked
Nobody Blocked
R=C=1
All empty seats

Sample Cases	Bounds
Sample Cases (3x)	2 ≤ R, C ≤ 5
Large / TLE Cases (1x)	R, C = 430
Random/Edge Cases (19x)	2 ≤ R, C ≤ 25

What was learned